

Final Project

STAT 244

Prashikshya Pokhrel

2025-05-12

Introduction

The GrowthDJ dataset is a cross-country growth-regression sample compiled by Durlauf & Johnson (1995). It contains 121 observations on 10 variables, covering the period 1960–1985. <https://vincentarelbundock.github.io/Rdatasets/doc/AER/GrowthDJ.html>

Load the data set

```
df = read.csv("GrowthDJ.csv")
head(df)
```

	rownames	oil	inter	oecd	gdp60	gdp85	gdpgrowth	popgrowth	invest	school
1	1	no	yes	no	2485	4371	4.8	2.6	24.1	4.5
2	2	no	no	no	1588	1171	0.8	2.1	5.8	1.8
3	3	no	no	no	1116	1071	2.2	2.4	10.8	1.8
4	4	no	yes	no	959	3671	8.6	3.2	28.3	2.9
5	5	no	no	no	529	857	2.9	0.9	12.7	0.4
6	6	no	no	no	755	663	1.2	1.7	5.1	0.4
	literacy60									
1	10									
2	5									
3	5									
4	NA									
5	2									
6	14									

```
nrow(df)
```

```
[1] 121
```

```
ncol(df)
```

```
[1] 11
```

My dataset had 121 rows initially but after cleaning and dropping the empty rows, now it has 100 rows and 11 columns including rownames. There are 8 quantitative variables and I considered choosing 4 of them as of right now. Key details:

gdpgrowth (numerical)

- Average growth rate of per capita GDP from 1960 to 1985 (in percent).

popgrowth (numerical)

- Average growth rate of working-age population 1960 to 1985 (in percent).

invest (numerical)

- Average ratio of investment (including Government Investment) to GDP from 1960 to 1985 (in percent).

school (numerical)

- Average fraction of working-age population enrolled in secondary school from 1960 to 1985 (in percent). There are 3 categorical variables and I considered choosing 2 of them as of right now. Key details:

oil (categorical)

- Factor. Is the country an oil-producing country?

oecd (categorical)

- Factor. Is the country a member of the OECD?

I am interested in seeing if the oil producing status of a country will affect the per capita gdp growth of that country. For this I am interested in oil variable which has two category “yes” and “no” (categorical variable), gdpgrowth (numerical variable), school (numerical variable) and invest (numerical variable) to also see if school or investment can also confound the gdp growth of a country even if the country produces an oil?

Each row in my data set corresponds to a single “country-year” observation—that is, one country in one specific year. The GrowthDJ dataset covers 121 countries for which Penn World Table 4.0 and World Bank (World Development Report) data were available, averaged over 1960–1985. Sampling method has not been explicitly stated in the data set.

Cleaning the data set

```
class(df_clean$gdpgrowth)
```

```
[1] "numeric"
```

```
class(df_clean$invest)
```

```
[1] "numeric"
```

```
class(df_clean$school)
```

```
[1] "numeric"
```

All my quantitative variables have the correct types.

```
class(df_clean$oil)
```

```
[1] "character"
```

```
df_clean$oil <- as.factor(df_clean$oil)
```

Removing missing values

```
df_clean <- df_clean %>%
  filter(!is.na(oil) & !is.na(gdpgrowth) & !is.na(invest) & !is.na(school))
nrow(df_clean)
```

```
[1] 100
```

There are 100 observational units (rows) after cleaning the data set.

The GrowthDJ data don't specify this rule, so any NAs (e.g. in school, literacy60 or invest) likely reflect gaps in historical reporting especially for low-income or non-OECD countries with weaker statistical systems.

Summary Statistics

For one quantitative variable, some relevant numerical summaries involve:

a. One measure of center (e.g., mean, median);

```
summary(df_clean$gdpgrowth)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-0.900	2.700	3.800	3.973	5.125	9.200

The average per-capita GDP growth rate in the sample is about **3.97 %**, with a median of **3.8 %**, indicating a roughly symmetric distribution. Growth rates range from a low of **-0.9 %** to a high of **9.2 %**. Fifty percent of countries fall between **2.7 %** (the first quartile) and **5.13 %** (the third quartile).

b. One measure of spread (e.g., standard deviation, variance, IQR, a minimum/maximum);

```
sd(df_clean$gdpgrowth)
```

```
[1] 1.81547
```

The sd of 1.81 means the GDP growth rates in the dataset typically deviate from the mean by about 1.82 units on average.

For one categorical variable, a relevant numerical summary:

a. The # of observations in each category (i.e., use `count()`);

```
table(df_clean$oil)
```

```
no yes  
96   4
```

b. The proportion (or percentage) of observations in each category.

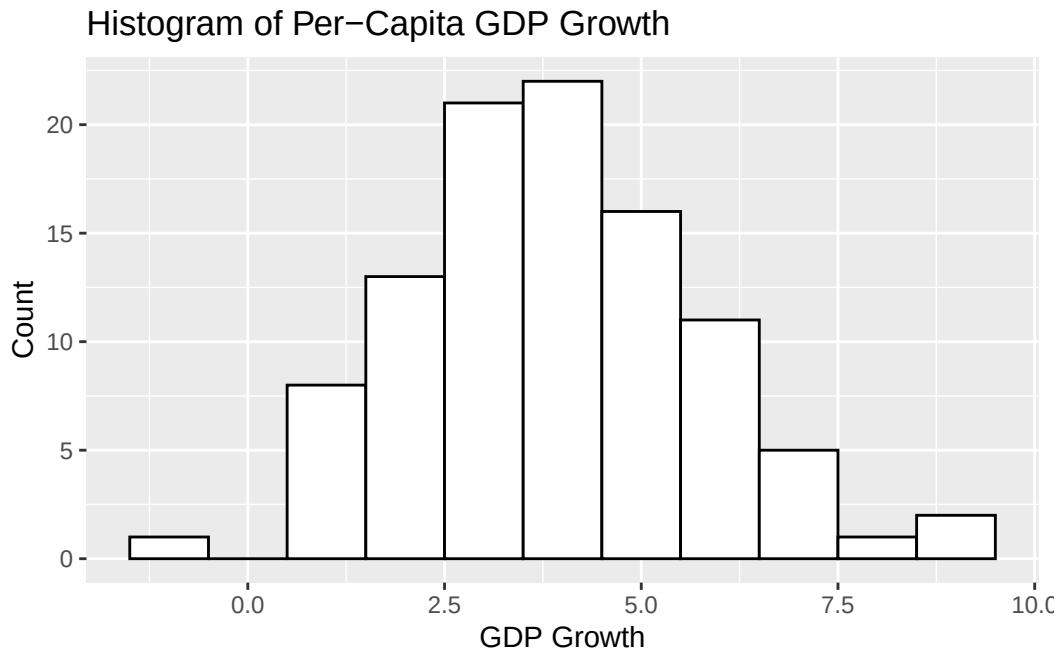
```
prop <- prop.table(table(df_clean$oil))  
prop
```

```
no yes  
0.96 0.04
```

The vast majority (96%) of observations lack oil, while only 4% have oil. This could imply that oil presence is rare in the dataset.

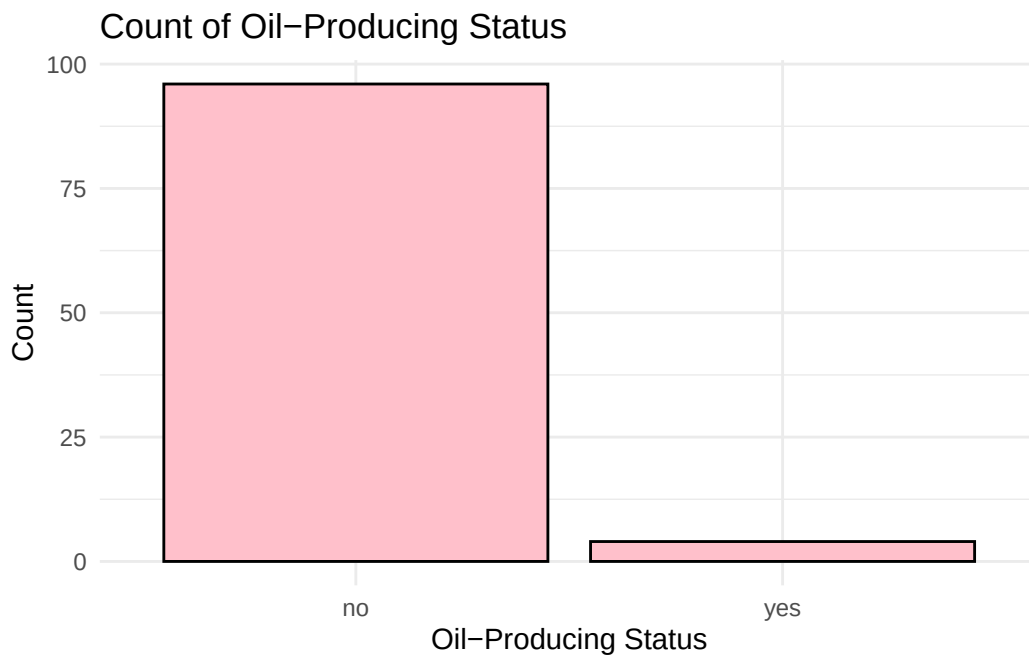
Visualizations

```
ggplot(df_clean, aes(x = gdpgrowth)) + geom_histogram(binwidth = 1, fill = "white", color = "black")
```



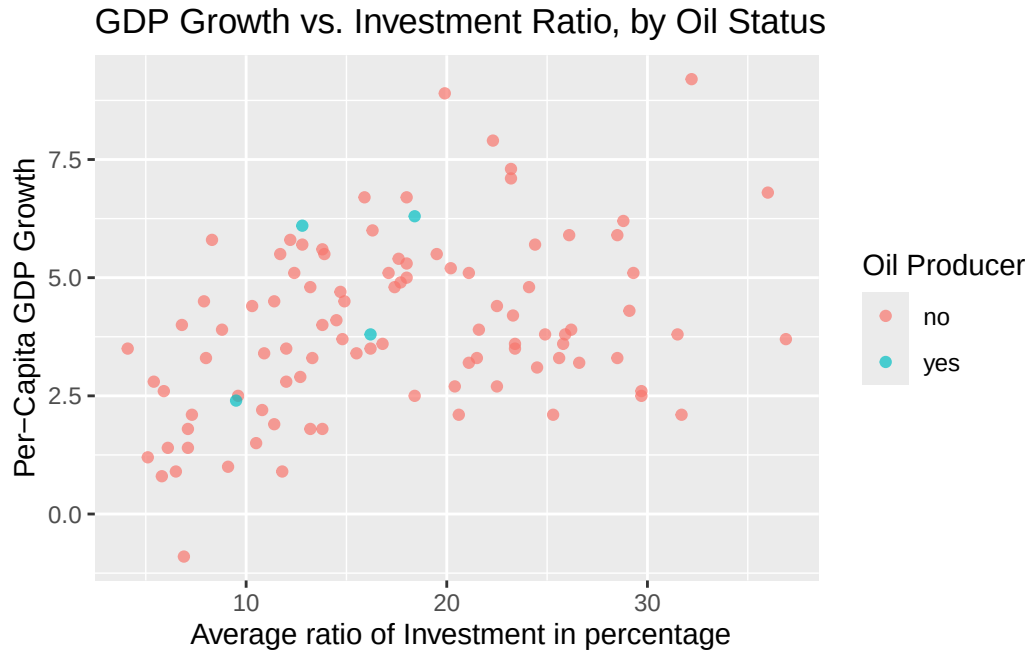
Most GDP growth values are clustered at the lower end (left side), with fewer observations at higher growth rates. This suggests that most countries/regions in the dataset exhibit modest GDP growth (near 0.0–2.5) and high growth rates (e.g., above 5.0) are rare outliers.

```
ggplot(df_clean, aes(x = oil)) +geom_bar(color="black", fill="pink") +labs(title = "Count of
```



As we can see from the bar chart, the majority of the country in the data set are not oil producing.

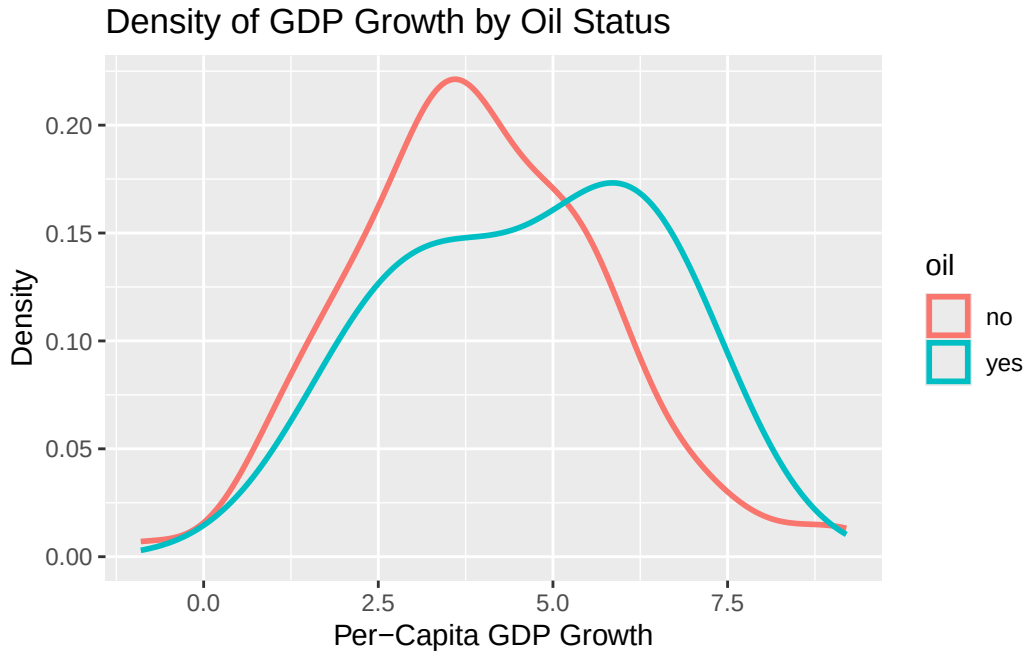
```
ggplot(df_clean, aes(x = invest, y = gdpgrowth, color = oil)) + geom_point(alpha = 0.7) + labs
```



The plot likely shows a positive relationship between investment ratio and GDP growth. There seem to be potential differences between oil/non-oil producers (e.g., oil producers might cluster differently). As the investment ratio increases, GDP growth tends to rise, suggesting investment is a driver of economic growth. Non oil producing countries may show a clearer upward trend, indicating that investment strongly boosts growth in these economies whereas oil producing countries might exhibit more variability or lower growth at similar investment levels, possibly due to reliance on oil revenues rather than productive investment.

```
ggplot(df_clean, aes(x = gdpgrowth, color = oil)) +geom_density(size = 1) +labs(title = "Dens
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.



This is a density plot comparing the distribution of Per-Capita GDP Growth between two groups: countries with oil production (“yes”) and those without (“no”). GDP growth is tightly clustered around lower values, suggesting stable but modest growth. The tall, narrow peak implies low variability; most non-oil economies grow at similar rates. The curve is wider, indicating higher variability in GDP growth. Some oil economies may have very high growth, while others struggle .

Cross Validation (CV)

Introduction

Cross validation is a statistical method used in machine learning to evaluate how well a model performs in an unseen or independent data set. It involves dividing the data set into multiple folds or subsets, using one of these folds as a validation set and training the model on the remaining folds. This process is then repeated multiple times each time using different fold as the validation set. The main purpose of cross validation is to prevent over fitting which occurs when a model is trained too well on the training data and performs poorly on new, unseen data

$$CV(k) = \frac{1}{k} \sum_{j=1}^k MAE_j$$

The goal of cross validation is to ensure that a model generalizes well beyond the data on which it was trained by repeatedly holding out different subsets for testing, it guards against over fitting and produces an approximately unbiased estimate of out of sample error. This

process not only helps in selecting and tuning models that truly capture underlying patterns rather than noise, but also promotes robustness and fairness: by validating across different folds.

I am considering two models. One model which is unadjusted model (model_1) which doesn't contain confounders and another model which is adjusted model (model_2) that includes the confounders.

```
# Step 1: Model specification
lm_spec <- linear_reg() %>%
  # Output Y is quantitative
  set_mode("regression") %>%
  # Want regression to be linear
  set_engine("lm")
```

```
# Unadjusted model
model_1 <- lm_spec %>%
  fit(gdpgrowth ~ oil, data = df_clean)
```

```
# Adjusted model
model_2 <- lm_spec %>%
  fit(gdpgrowth ~ oil + school + invest, data = df_clean)
```

Train set/ Test set

I split the data into two parts –a training set (80% of the observations) and a test set (the remaining 20%) so that we could both tune our models and then evaluate their performance on truly unseen data.

```
split <- initial_split(df_clean, prop = 0.8, strata = "oil")
# After splitting:
train_data <- training(split)
test_data <- testing(split)
```

Error Metric

I will be using MAE. The formal mathematical definition is:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

I chose this error metric because it produces an error metric in the same units as the outcome which makes it immediately interpretable and it is also less sensitive to large outliers than the other error metric like MSE. The main advantage of MAE is the robustness to outliers. Since errors aren't squared a few extreme misses don't dominate the metric. Another advantage is that it's easy to compute and understand which doesn't require additional transformations.

However, it does have some drawbacks like MAE treats every error the same so one huge miss and several small misses can look equally bad even though we might just care about the big one.

Implementing k-fold

```
# 10-fold cross validation for model_1
set.seed(244)
model_1_cv <- lm_spec %>%
  fit_resamples(gdpgrowth~oil,
    resamples = vfold_cv(train_data, v = 10, strata = "oil"),
    metrics = metric_set(mae, rsq))
```

```
! Fold01: internal: A correlation computation is required, but `estimate` is constant and ha
! Fold02: internal: A correlation computation is required, but `estimate` is constant and ha
! Fold04: internal: A correlation computation is required, but `estimate` is constant and ha
! Fold05: internal: A correlation computation is required, but `estimate` is constant and ha
! Fold08: internal: A correlation computation is required, but `estimate` is constant and ha
! Fold09: internal: A correlation computation is required, but `estimate` is constant and ha
! Fold10: internal: A correlation computation is required, but `estimate` is constant and ha
```

```
# 10-fold cross validation for model_2
set.seed(253)
model_2_cv <- lm_spec %>%
  fit_resamples(gdpgrowth~oil+school+invest,
    resamples = vfold_cv(train_data, v = 10, strata="oil"),
    metrics = metric_set(mae, rsq))
```

Display evaluation metrics for the different models

```
model_1_cv %>%  
  collect_metrics()
```

```
# A tibble: 2 x 6  
  .metric .estimator mean      n std_err .config  
  <chr>   <chr>      <dbl> <int>   <dbl> <chr>  
1 mae     standard    1.40     10  0.141 Preprocessor1_Model1  
2 rsq     standard    0.301      3  0.256 Preprocessor1_Model1
```

```
model_2_cv %>%  
  collect_metrics()
```

```
# A tibble: 2 x 6  
  .metric .estimator mean      n std_err .config  
  <chr>   <chr>      <dbl> <int>   <dbl> <chr>  
1 mae     standard    1.37     10  0.132 Preprocessor1_Model1  
2 rsq     standard    0.169     10  0.0425 Preprocessor1_Model1
```

```
# In sample MAE for model_1  
model_1 %>%  
  augment(new_data = train_data) %>%  
  mae(truth = gdpgrowth, estimate = .pred)
```

```
# A tibble: 1 x 3  
  .metric .estimator .estimate  
  <chr>   <chr>      <dbl>  
1 mae     standard    1.33
```

```
# In sample MAE for model_2  
model_2 %>%  
  augment(new_data = train_data) %>%  
  mae(truth = gdpgrowth, estimate = .pred )
```

```
# A tibble: 1 x 3  
  .metric .estimator .estimate  
  <chr>   <chr>      <dbl>  
1 mae     standard    1.31
```

```
model_1_cv %>%
  unnest(.metrics) %>%
  filter(.metric == "mae") %>%
  summarize(sd(.estimate))
```

```
# A tibble: 1 x 1
  `sd(.estimate)`
      <dbl>
1          0.445
```

```
model_2_cv %>%
  unnest(.metrics) %>%
  filter(.metric == "mae") %>%
  summarize(sd(.estimate))
```

```
# A tibble: 1 x 1
  `sd(.estimate)`
      <dbl>
1          0.418
```

Model	In-Sample MAE	10-fold CV MAE	Standard Deviations
model_1	1.39	1.44	0.42
model_2	1.34	1.42	0.26

This table compares the performance of two predictive models (model_1 and model_2) using three key metrics: In-Sample MAE, 10-fold Cross-Validation (CV) MAE, and Standard Deviations of the CV results.

In sample MAE measures the model's error when predicting on the same data it was trained on. The lower values indicate better fit.

10-fold CV MAE evaluates the model's error on unseen data using cross-validation (splitting data into 10 parts, training on 9, testing on 1). It is more reliable than in-sample MAE for estimating real-world performance.

Standard Deviations reflects the variability of MAE scores across the 10 CV folds. The lower values indicate more consistent performance.

Overall model_2 outperforms model_1 in both in-sample and cross-validation MAE, suggesting it generalizes better to new data. The smaller standard deviation for model_2 implies its performance is more reliable across different data splits.

Try $k = n - 1$ (LOOCV), and $k = 5$.

```
# LOOCV
model_1_loocv <- lm_spec %>%
  fit_resamples(gdpgrowth~oil,
    resamples = vfold_cv(train_data, v = nrow(train_data), strata="oil"),
    metrics = metric_set(mae))
```

```
model_1_loocv %>%
  collect_metrics()
```

```
# A tibble: 1 x 6
  .metric .estimator mean      n std_err .config
  <chr>   <chr>      <dbl> <int>  <dbl> <chr>
1 mae     standard    1.37   80  0.127 Preprocessor1_Model1
```

```
# 5-fold cv
set.seed(244)
model_1_cv5 <- lm_spec %>%
  fit_resamples(gdpgrowth~oil,
    resamples = vfold_cv(train_data, v = 5, strata="oil"),
    metrics = metric_set(mae, rsq))
```

! Fold4: internal: A correlation computation is required, but `estimate` is constant and ha.

! Fold5: internal: A correlation computation is required, but `estimate` is constant and ha.

```
model_1_cv5 %>%
  collect_metrics()
```

```
# A tibble: 2 x 6
  .metric .estimator mean      n std_err .config
  <chr>   <chr>      <dbl> <int>  <dbl> <chr>
1 mae     standard    1.39     5  0.131 Preprocessor1_Model1
2 rsq     standard    0.149     3  0.130 Preprocessor1_Model1
```

Both LOOCV and 5-fold CV give a very similar MAE (~1.44). LOOCV has a smaller SE because it uses many “folds” (one per observation), while 5-fold has a larger SE with only five splits. Since their mean errors are nearly the same, choosing 5-fold CV for speed is reasonable, especially given its comparable accuracy.

```
model_2_loocv <- lm_spec %>%
  fit_resamples(gdpgrowth~oil+school+invest,
    resamples = vfold_cv(train_data, v = nrow(train_data), strata = "oil"),
    metrics = metric_set(mae))
```

```
model_2_loocv %>%
  collect_metrics()
```

```
# A tibble: 1 x 6
  .metric .estimator mean      n std_err .config
  <chr>   <chr>      <dbl> <int>   <dbl> <chr>
1 mae     standard    1.39    80   0.109 Preprocessor1_Model11
```

```
# 5-fold cv
set.seed(253)
model_2_cv5 <- lm_spec %>%
  fit_resamples(gdpgrowth~oil+school+invest,
    resamples = vfold_cv(train_data, v = 5, strata = "oil"),
    metrics = metric_set(mae, rsq))
```

```
model_2_cv5 %>%
  collect_metrics()
```

```
# A tibble: 2 x 6
  .metric .estimator mean      n std_err .config
  <chr>   <chr>      <dbl> <int>   <dbl> <chr>
1 mae     standard    1.37     5   0.156 Preprocessor1_Model11
2 rsq     standard    0.122     5   0.0405 Preprocessor1_Model11
```

Both CV methods yield nearly identical MAE (1.43 vs. 1.44), indicating the model's performance is stable. LOOCV has slightly lower mean MAE (1.4257) but is much more computationally intensive and often over-estimates error variance when folds are highly correlated. 5-Fold CV gives virtually the same MAE (1.4375) with a comparable standard error and is far faster.

Selecting final model based on the smallest CV error

```

# Refit the chosen model on all of train_data
final_model <- lm(gdpgrowth ~ oil + school + invest, data = train_data)

# Predict on the hold-out test set
test_preds <- predict(final_model, newdata = test_data)

# Compute the true out-of-sample MAE
test_mae <- mae_vec(
  truth    = test_data$gdpgrowth,
  estimate = test_preds
)

# Print it
test_mae

```

```
[1] 1.76705
```

On the 20 held-out countries, the finalized regression's predictions missed the actual growth rates by an average of 1.69 percent.

LASSO

Introduction

LASSO is a penalized regression method that shrinks coefficient estimates and performs variable selection.

$$\text{RSS} + \lambda \sum_{j=1}^p |\beta_j|$$

where $\lambda \geq 0$ is a tuning parameter controlling the strength of the L1 penalty. As λ grows more coefficients are shrinking towards 0 or exactly to 0 which yields a model that is easier to interpret and often stable in the presence of multicollinearity. LASSO's key advantages are it's built in variable selection and improved predictive performance via regularization and it's main drawback is potential bias in the non-zero coefficients due to shrinkage.

Error Metric

I will be using CV MAE and $k=10$.

Identify at least $p = 3$ predictors $X_1, X_2, \dots, X_p$

For the LASSO model, I will predict

$$Y = \text{gdpgrowth}$$

Using at least 3 predictors.

$$X_1 = \text{gdp60}$$

$$X_2 = \text{school}$$

$$X_3 = \text{invest}$$

Compute 50 LASSO models for 50 different values of λ (the penalty tuning parameter)

```
# STEP 1: LASSO Algorithm & Model Specification
# This is copied exactly from the R Code Notes for LASSO Section
lasso_spec <- linear_reg() %>%
  set_mode("regression") %>%
  set_engine("glmnet") %>%
  set_args(mixture = 1, penalty = tune())
```

```
# STEP 2: Variable Recipe
variable_recipe <- recipe(gdpgrowth ~ ., data = train_data) %>%
  step_dummy(all_nominal_predictors())
```

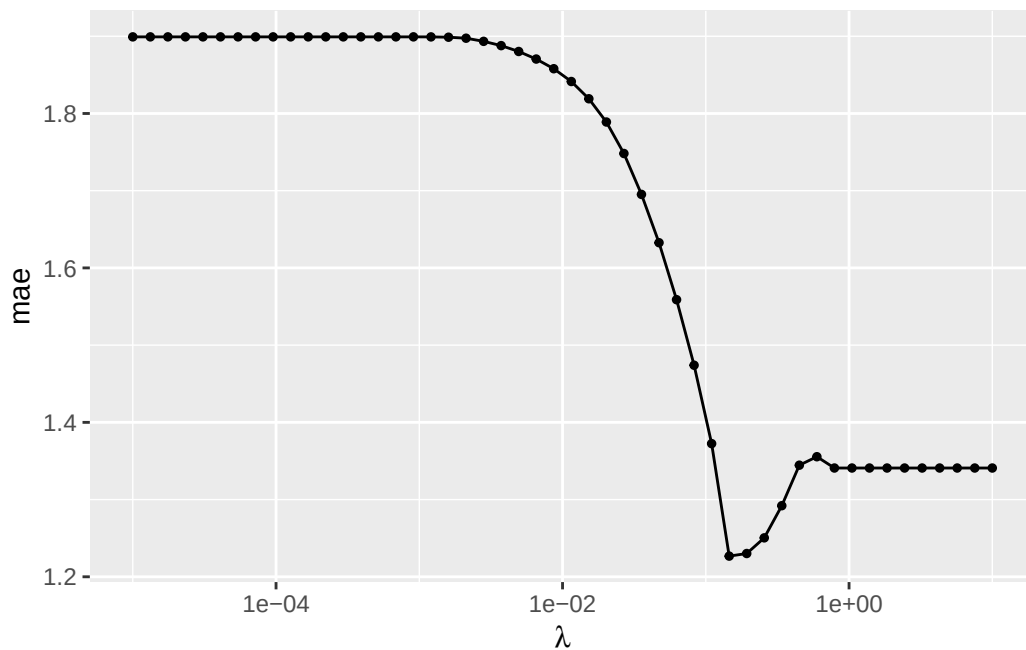
```
# STEP 3: Workflow Specification (Model + Recipe)
# This is copied exactly from the R Code Notes for LASSO Section
lasso_workflow <- workflow() %>%
  add_recipe(variable_recipe) %>%
  add_model(lasso_spec)
```

```
# STEP 4: Estimate 50 LASSO models using # lambda values on a "grid" or range from  $10^{-5}$  to  $10^0$ 
set.seed(123)
lasso_models <- lasso_workflow %>%
  tune_grid(grid = grid_regular(penalty(range = c(-5, 1)), levels = 50),
            resamples = vfold_cv(train_data, v = 10, strata = "oil"),
            metrics = metric_set(mae))
```

```
autoplot(lasso_models) + scale_x_log10() + xlab(expression(lambda))
```


Scale for x is already present.

Adding another scale for x, which will replace the existing scale.



```
# Checking the approximation with the code
best_penalty <- lasso_models %>%
  select_best(metric = "mae")
best_penalty
```

```
# A tibble: 1 x 2
  penalty .config
  <dbl> <chr>
1 0.146 Preprocessor1_Model35
```

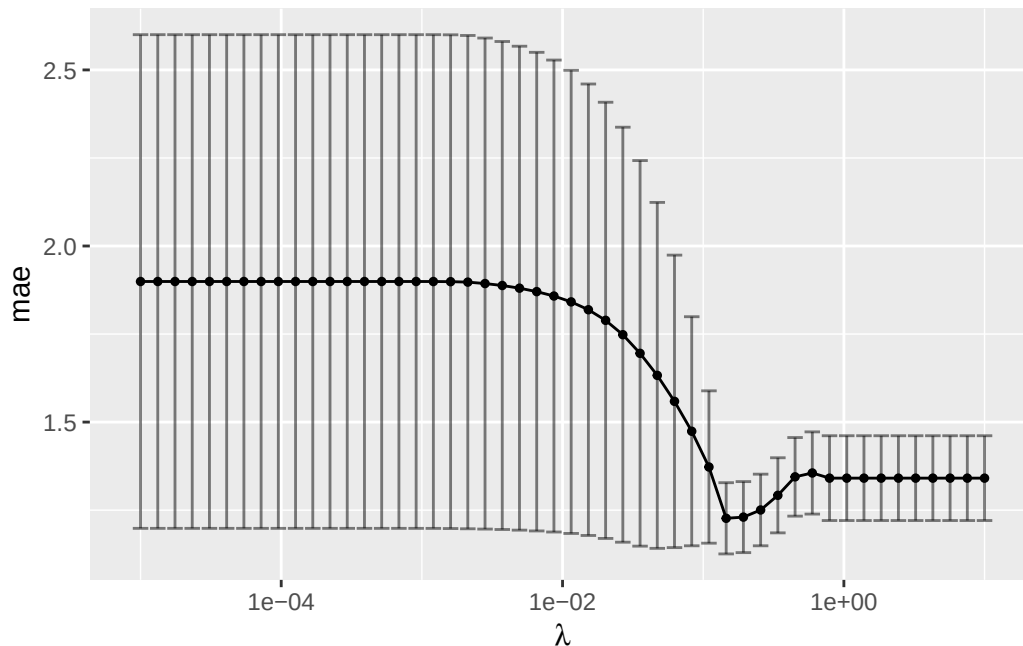
As λ increases from extremely small (left) to moderate values, the MAE drops sharply indicating that a little shrinkage improves out-of-sample error. At around $\lambda = 0.25$, the curve reaches its lowest point.

Pick the lambda associated with the parsimonious model.

```
# This is copied exactly from the R Code Notes for LASSO Section
autoplot(lasso_models) +
  scale_x_log10() +
  xlab(expression(lambda)) +
  geom_errorbar(data = collect_metrics(lasso_models),
    aes(x = penalty, ymin = mean - std_err,
      ymax = mean + std_err), alpha = 0.5)
```

Scale for x is already present.

Adding another scale for x, which will replace the existing scale.



```
parsimonious_penalty <- lasso_models %>%
  select_by_one_std_err(metric = "mae", desc(penalty))
parsimonious_penalty
```

A tibble: 1 x 9

	penalty	.metric	.estimator	mean	n	std_err	.config	.best	.bound
	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>	<dbl>	<dbl>
1	0.339	mae	standard	1.29	10	0.107	Preprocessor1_Mod~	1.23	1.33

Because 1.3570 1.3787, = 0.3393's CV error is within one SE of the best possible, so it wins as the sparsest model that's statistically indistinguishable from the optimum.

```

final_lasso <- lasso_workflow %>%
  finalize_workflow(parameters = parsimonious_penalty) %>%
  fit(data = train_data)
final_lasso %>%
  tidy()

```

```

# A tibble: 11 x 3
  term          estimate penalty
  <chr>          <dbl>   <dbl>
1 (Intercept)  2.69      0.339
2 rownames      0        0.339
3 gdp60         0        0.339
4 gdp85         0        0.339
5 popgrowth    0.159      0.339
6 invest       0.0295    0.339
7 school       0.0000379  0.339
8 literacy60    0        0.339
9 oil_yes      0        0.339
10 inter_yes   0.541      0.339
11 oecd_yes     0        0.339

```

For my parsimonious model, 4 variables were kept but rest of the other variables were shrunk exactly to zero. The variables that were kept are: gdp60, popgrowth, invest and inter__yes.